Participant's Section

I. Use Case Short Title

Physical Records Management System (PRMS)

II. Problem Statement & Background

Bank operations in India generate a large volume of paper records — client KYC packs, loan agreements, accounting records, internal and external audit files, vendor invoices and tax records. These documents are filed into cartons and shipped to offsite warehouses operated by third-party storage vendors.

The estate today is approximately 50,000+ physical cartons across 6 locations in India, operated by 3 vendors, and serving 3 business entities and 13 departments. It is currently managed through emails, spreadsheets and separate vendor portals. There is no single source of truth, which creates four concrete failures:

- Traceability — no reliable end-to-end trail of a carton or file from creation to destruction.
- Compliance — retention expiry is tracked manually, so destruction is late, missed, or (worse) performed on records under legal hold.
- Operational efficiency — inventory data sits with the vendor and the reconciliation of vendor inventory, retrieval logs and invoices against business data is manual today.
- Audit — producing real-time, auditable evidence of where a document is, and proof of its destruction, takes days.

III. Objective

Build a Physical Records Management System that provides end-to-end visibility of the physical record lifecycle, automates retention compliance, reconciles internal records against vendor declarations, monitors vendor performance and cost, and maintains an auditable trail of every state change.

In one line

One system of record for every carton, every movement, every rupee, and every disposal — with the exceptions surfaced, not buried.

IV. Definition of Done

A submission is complete when all the following can be demonstrated on the supplied dataset.

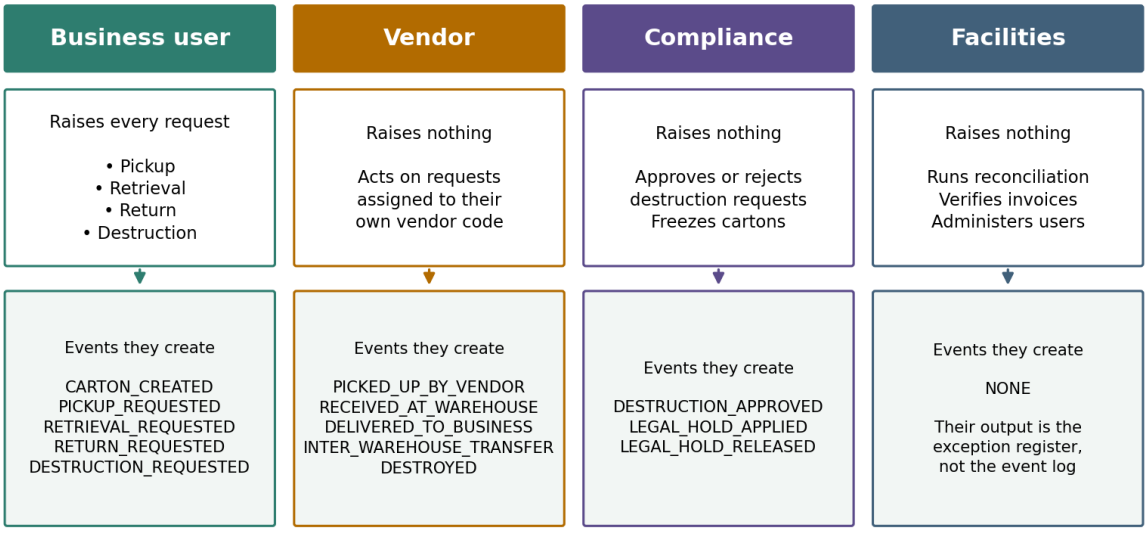
#	Acceptance criterion	How it is verified
1	Centralized dashboard for all 4 business roles showing inventory by entity, department, location, vendor and status.	A facility can login filter data any entity. A business user login sees only its own department.

#	Acceptance criterion	How it is verified
2	UI and workflow covering carton creation, pickup, retrieval, return and destruction request	A full lifecycle driven end-to-end live by four logged-in roles: business raises, vendor confirms, compliance approves destruction, facilities monitor and reconciles.
3	Append-only event log recording every movement, confirmation, approval, hold and destruction.	Any carton's full history is shown with actor, timestamp and request reference ID
4	Retention engine deriving destruction due dates from policy reference data and raising alerts.	Change one row in the policy table; due dates and alerts recompute without a code change.
5	Legal hold blocking destruction.	A judge attempts to destroy a carton under an active hold. The server refuses and shows the matter reference.
6	Reconciliation of the vendor's inventory declaration against internal records.	A facilities login runs the Q4-2025 or Q2-2026 cycle: 'matched', 'missing-at-vendor' and 'unknown-to-us counts' are reported, and each break persists as an exception with severity and an owner
7	Role-based access control with four distinct roles — business user, facilities team, vendor and compliance officer.	A vendor login cannot see cost data or another vendor's requests; a business user cannot see another department; facilities read across all entities but cannot alter the event log; compliance can hold and approve but cannot raise a destruction request.

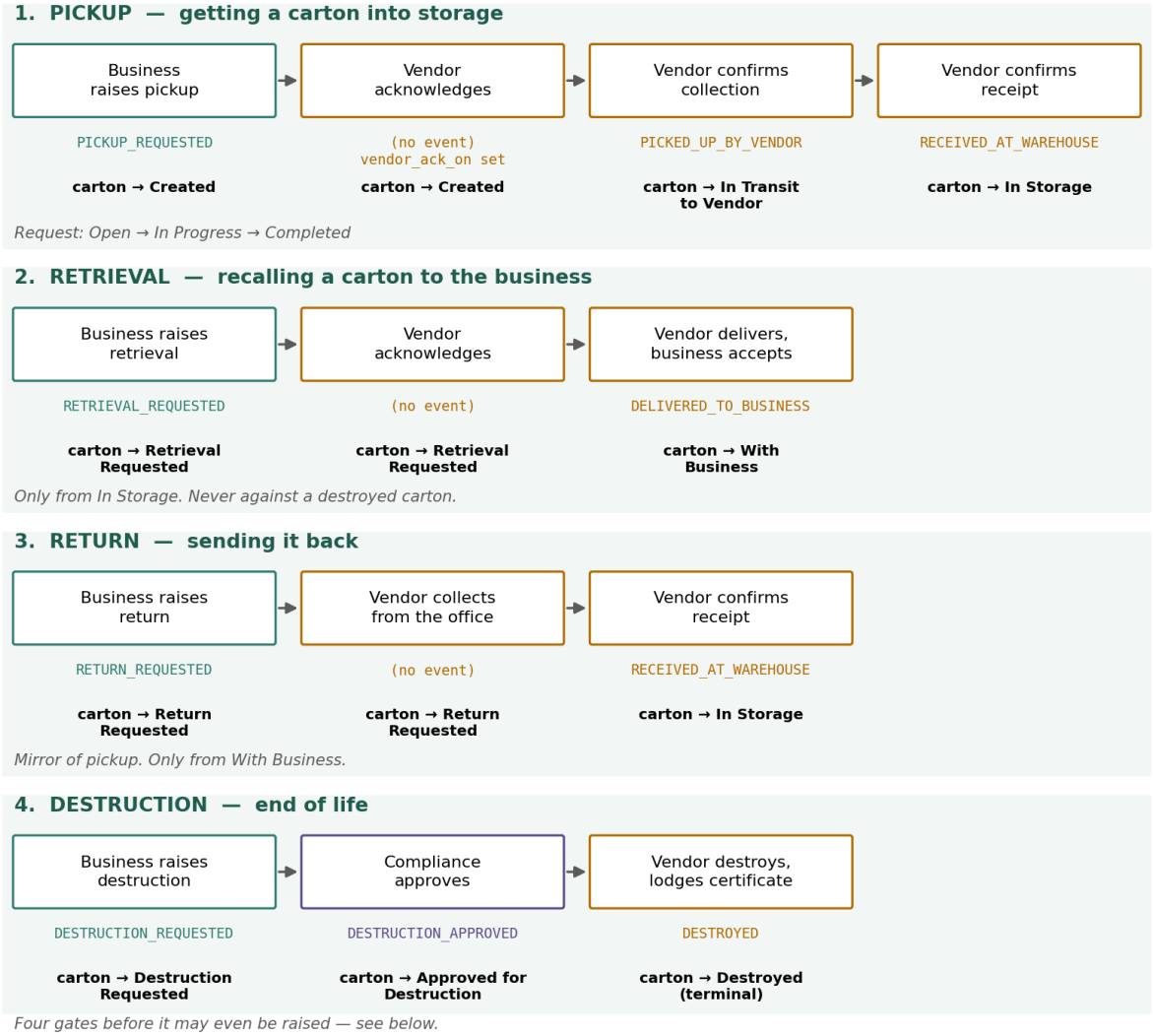
V. Expected Outcomes

Four roles each contributing to PRMS in different capacities

Only the business user raises requests; the vendor acts on them, compliance gates destruction, and facilities watches the whole estate without touching it and performs reconciliation. What each role must be able to do, and the rule attached to each action, is set out in Workflows 1-4 below.



Every event names the carton it happened to, the actor who caused it, and the request it fulfils.



The event log is the source of truth

This is the single most important architectural decision in the system.

Every fact about a carton is established by an event and by nothing else - its status, its location, whether it is billable this month, whether it may be destroyed. A status column is a cached answer, not the answer. If the two disagree, the log wins, and your system should be able to say so.

- One log, not two. Creation, requests, approvals, holds and corrections all append here. A second audit table drifts from the first.
- Append-only. No UPDATE, no DELETE. A mistake is corrected by appending a compensating event that references the original.
- Every event records what happened, to which carton, when, by which actor in which role, under which request, and through which channel (UI, upload, API, scheduler).

- Reconciliation, cost and retention all read the log. **The vendor's declaration is compared against what the log says we believed on the declaration date, not what we believe today**.
- **Storage is billed for the months the log says the carton was in a warehouse**.

Rules the service layer must enforce

Judges will test these by trying to break them. Each must be enforced on the server, and a refusal must return a clear reason.

Area	Rule
Retention	The destruction due date is the retention trigger date plus the retention period, looked up in ref_retention_policy by document type and archival policy. Never hardcode it; never let a user enter it. *Where a carton has no trigger date, raise a data-quality exception rather than substituting a default.*
Legal hold	While a hold is active, destruction is blocked however long retention has expired, and the refusal must state the matter reference. A hold restricts destruction only: the carton may still be recalled, returned and moved.
Destruction	A destruction request is valid only when all four hold: retention has expired, no legal hold is active, the carton is in storage. Approval rests with Compliance. Once destroyed, no further action is possible.
Status	A carton's status is never supplied by the client. It is derived from its events and written in the same transaction as the event that caused it. The actions offered must follow from the current status; any other is refused.
SLA	Turnaround is measured in business days, excluding weekends and the holidays in ref_holiday_calendar. ref_sla defines which pair of timestamps applies. The destruction clock starts at approval, not at the request. Breach is derived by comparing completion against the due date; a request still open past its due date has also breached.
Cost	Monthly storage cost is the cartons held that month times the storage rate in force that month. Rates are effective-dated and one vendor's rate changes partway through the data, so use the rate applicable to the service period. Activity cost is the pickups, retrievals, returns and destructions in the month at their rates. Tax is 18 per cent
Access	A business user sees only their own department. A vendor user sees only requests issued to their own vendor code, and never cost figures or carton contents. A compliance officer applies holds and approves destructions. Facilities reads across all entities and administers access, read but cannot alter the event log.

VI. Solution Requirements (Good to Have)

This section is the technical specification. Section VI.B is the functional capability: what the system must do, without prescribing how you build it. Everything below is verifiable at demo time.

Recommended technology stack — not mandatory

Any language, framework and database is fine. The list below is a safe default, so no time is lost debating one and mentors can help you. If you choose differently, say so and why in your README.

Layer	Recommended
User interface	React (Vite) or Angular — a simple UI that works beats a half-built SPA
Backend	Java 17 + Spring Boot, or Python 3.11 + FastAPI
Database	A lightweight PostgreSQL Database Server hosted on your local machine or even an SQLite (file-based database) works for a single-node demo
Auth	Simple user_id/password-based authentication (ensuring hashing of password in storage) For authorisation: JWT carrying role/claims. Using sessions is fine too, provided authorisation is checked server-side
Jobs, reports, ingestion	Any scheduler that re-runs without duplicate alerts; server-side PDF; any XLSX reader that reports rejected rows

Golden Rule: Keep HTTP handling, business rules and data access separate

Software Engineering Excellence Checklist

Area	Expectation
Version Control	Code maintained in Git with meaningful commits.
Dependencies	Reproducible build using pinned dependency versions.
Configuration & Secrets	No hard-coded credentials or API keys.
Code Quality	Readable, modular, maintainable code.
Error Handling	Graceful handling of invalid inputs and failures.
Logging	Basic operational logging for troubleshooting.
Testing	Demonstrate positive and negative test scenarios.
Documentation	README explaining setup, execution, architecture, and assumptions.
Build & Run	Solution can be installed and executed using documented steps.
Final Validation	Team demonstrates a clean working build during evaluation.

VII. Detailed breakdown of Tasks

Attempt these in order start with Basic tasks then move to Intermediate and finally attempt Advance tasks.

i. Basic Tasks — Must Have

Task	What good looks like
a. Build the data model for all aspects of this use case using the supplied dataset	Normalised schema with referential integrity policy held as data. Use the supplied data set to load the data to your database. Ship the DDL and a diagram.
b. Develop UI and workflow	Business creates a carton, raises pickup request. Vendor answers business requests. Facilities create users and assign roles, entities and departments. Server-side authorisation on every endpoint.
c. Create a dashboard for each user role showing total volumes by entity and location	Filterable, drillable to the carton list, and exportable. Counts must tie back to the source data.

Task	What good looks like
d. Load the supplied dataset	Inventory, files, requests, event log and reference sheets seeded by a repeatable script. Vendor artifacts load into their own tables, never merged into the carton master. Rejected rows are reported, not dropped.

ii. Intermediate Tasks — Must Have

Start with the workflows below, role by role.

Each user role holds a different responsibility. The "rule" column is what your backend service layer logic must enforce.

Flow 1 — Business user (record owner)

Step	What they do	Rule that must hold
1	Register a new carton: select the document type, archival policy, retention trigger date	The destruction due date is derived from the policy table, never typed in
2	Index the files inside the carton	Add files to a carton. One carton has **files belonging to the same document type**
3	Request a pickup and hand the carton to the vendor	The carton is not in storage until the vendor confirms it's received at the warehouse. Pickup alone is not enough.
4	Retrieve a carton back when the business needs it back	Only a carton in storage can be recalled. A destroyed carton cannot be recalled at all.
5	Request return once finished with it	A carton held beyond the configured threshold raises an alert to the owner
6	Act on a retention alert and raise a destruction request	A carton held beyond the configured threshold raises an alert to the owner.

Flow 2 — Facilities team

Step	What they do	Rule that must hold
1	Dashboard containing cartons stored per entity, department, location and vendor	Cross-entity read. Figures must tie back to the underlying data.
3	Ingest the vendor's periodic inventory declaration and reconcile it – inventory recon.	Stored as received. Re-uploading the same file must not duplicate anything. Any breaks recorded during a recon should persist as exceptions with an owner
4	Compute monthly storage and activity cost and verify the vendor invoice – invoice recon.	Compared line by line against independently computed cost. Any over or under charging is raised as dispute

Step	What they do	Rule that must hold
5	Maintain vendors, locations, rates and SLA reference data	Rates are effective-dated. Changing one must change future billing without touching code.
6	Administer user access — create users, assign role, entity and department scope	A user has one role and one scope; vendor users carry a vendor code and no department. Deactivating a user removes their access but never their authorship of past events.

Workflow 3 — Vendor

Step	What they do	Rule that must hold
1	See the requests assigned to them	Own vendor code only. Never another vendor's work, never carton contents, never rates or cost.
2	Acknowledge a request	The acknowledgement timestamp is theirs, it puts the Request ID to be 'In Progress'
3	Confirm collection, then confirm receipt at the warehouse	Two distinct confirmations. **The gap between them is where cartons go missing.**
4	Deliver a recalled carton, and collect it again on return	Delivery is confirmed against the destination the request specified.
5	Confirm destruction and record the certificate reference	A certificate reference is never reused across batches, dates or vendors.

Workflow 4 — Compliance officer

Step	What they do	Rule that must hold
1	Apply a legal hold to a carton, with a matter reference and reason	A hold overrides retention absolutely, however long expired the carton is.
2	Approve or reject destruction requests	Never a request they raised themselves. Blocked outright if a hold is active or the carton is not in storage.
3	Release a hold, recording date and reason	Only Compliance may release. The release is itself an audit record.
4	Read the full history of any carton across all entities	Read-only on the event history. Compliance may not alter what happened.

Facilities: Reconciliation

This is a required task, not an advanced one. Facilities logs in and runs reconciliation: it is a job with a person behind it, not a background calculation. The user picks a reconciliation and a period, runs it, and sees what it found. The run is recorded - who ran it, which period, and the counts it produced. Re-running the same period must change nothing; running a later period must age the existing open exception, not raise a second one.

Every reconciliation writes to ONE exception register. An exception carries its severity; evidence references and owner

Three reconciliations, each on its own artifact's cadence

Run	When it runs	Run parameter
1. Vendor recertification feed	Quarterly, when the vendor declares	The cycle, e.g. Q2-2026
2. Vendor invoice	Monthly, on receipt of the invoice	The invoice
3. Destruction certificates	Monthly sweep	The certificate, or a date range

Reconciliation 1 Breaks — Vendor recertification feed

Compares the vendor's quarterly declaration against what our books said on the declaration date. The feed is stored as received and never merged into the carton master: the disagreement is the product, so do not tidy its rows to match ours.

Break	Severity	Owner
We believe the carton is in storage, but the vendor did not declare it	High	Facilities
The vendor declared a carton we have never heard of	High	Facilities
The vendor says the carton is at a different warehouse than we do	Medium	Facilities

Reconciliation 2 Breaks — Vendor invoice

Runs in two passes. Check the invoice against itself first, then against cost we have computed independently.

Break	Severity	Owner
The invoice does not add up to its own lines	High	Vendor
A single monthly storage line is billing more than one month	Medium	Vendor
We have been charged more than the contract allows	Medium	Vendor
Activity has been charged with no instruction/reference behind it	Medium	Vendor
Storage is being charged for a carton that was already destroyed	High	Vendor
The invoice bills storage or activity that does not match the event log	Medium	Facilities

Reconciliation 3 Breaks— Destruction certificates

Matches the certificate against the event log, the destruction request and the hold register.

(A certificate legitimately covers a batch - one vendor, one month, many cartons - so a repeated certificate id is normal and must not be flagged)

Break	Severity	Owner
A carton was destroyed while a legal hold was in force	Critical	Compliance
One certificate reference has been used across different vendors or months	High	Vendor
A carton was destroyed but no certificate was ever lodged	High	Vendor
A certificate exists for a carton that was never destroyed	High	Facilities
Something happened to the carton after it was destroyed	High	Facilities
The certificate and the log disagree on when the carton was destroyed	Medium	Vendor

iii. Advanced Tasks — Good to Have

Task	What good looks like
a. NLP smart search	**"Find all 2022 tax records for Entity X" and similar. The query must translate to a constrained, parameterised query — never raw model-authored SQL. The user sees what it was interpreted as, results trace back to rows, and access control applies exactly as it does on screen.**
b. Stalled-state findings	**Three cases: a carton the vendor collected but never received into a warehouse; a carton the business borrowed and never sent back; and a request that is past its due date and still open. How long is "too long" comes from the ref_threshold sheet, never from your code - a judge will change 30 days to 7 and expect your counts to move. Flag only the ones past the threshold: a file borrowed last month is in use, not lost. Run it once a day, and do not raise the same finding twice.**
c. Vendor SLA and turnaround tracking	**Show how quickly each vendor finishes each kind of request, against the time the contract allows. Count working days only - skip weekends and the holidays listed in ref_holiday_calendar. The clock starts when the business raises the request, except for destruction where it starts when Compliance approves, and stops when the vendor confirms the work is done. A request still sitting open after its due date has missed its SLA too, not just one that finished late. Work out whether it breached when you show it; do not save it as a status. Break the numbers down by vendor and by activity and show the trend over the last six months.**

The dataset contains deliberate imperfections - some data-quality noise, some real operational failures. The break classes above find many of them; there are more. Judges score both detection and false positives, so flagging every row is not detection.

VIII. Input

A single workbook

- **prms_hackathon_dataset_v2.xlsx**, of 26 sheets. Start with README, then data_dictionary and column_dictionary - every column of every sheet, with its type, key role and meaning.
- **records_retention_policy_reference**: This policy governs the creation, storage, retrieval, recertification and disposition of physical records held

IX. Guardrails

Scope

- This is management and governance of physical records. Do not build a digital document viewer, a scanning pipeline, or an OCR tool.
- Vendor portals, courier tracking integrations and hardware (barcode scanners, RFID) are out of scope. You may simulate a scan as an API call.

Engineering

- Do not hardcode retention periods, rates, SLA values or holidays in code. If a judge cannot change a rule in configuration and see the behaviour change, the requirement is not met.
- Do not clean the dataset by editing the supplied file. Handle the imperfections in your pipeline — the imperfections are the exam.
- Do not fabricate additional data to make a feature look better. Extending the dataset with the supplied generator is allowed and encouraged; state clearly what is generated.

Complexity to respect

- The solution must handle multiple entities with different retention rules per entity and document type, and multiple vendors with different rates and SLAs.
- Dates in the dataset span a decade and some are missing. Timezone and date-only handling must be consistent.

.

X. Dataset

This is the description of the reference data set provided:

Sheet group	Sheets	Rows (approx.)
Guidance	README, data_dictionary, column_dictionary	200

Sheet group	Sheets	Rows (approx.)
Reference / master	ref_entities_departments, *ref_users_roles*, *ref_roles*, *ref_vendors*, *ref_locations*, *ref_document_types*, *ref_retention_policy*, *ref_rate_card*, *ref_sla*, *ref_holiday_calendar*, *ref_carton_status*, *ref_request_status*, ref_event_type, *ref_threshold*	180
Core inventory	*carton_inventory*, *file_index*, *legal_holds*	11,100
Operational	*requests*, *event_log*	15,000
Vendor	*vendor_recertification_feed*, *destruction_certificates*	4,800
Financial	*vendor_invoices, vendor_invoice_lines*	10,200

XI. Reference

- Records Management & Retention Policy (synthetic), supplied alongside this brief. Required reading: the retention engine, approval matrix and legal-hold rules are specified there, including the 180-day escalation threshold.
- Data privacy principles for document destruction - purpose limitation, storage limitation, evidence of disposal, and the primacy of legal hold over routine deletion.
- Three-way match as a reconciliation pattern (order, receipt, invoice). The same shape applies here to request, event and invoice - and to request, event and certificate.

Term	Meaning
Carton	A physical box of files. The unit of storage, billing and destruction.
DAP / entity / department	Every carton belongs to exactly one department.
Retention trigger date	The business event date retention counts from - loan closure, relationship closure, financial year end.
Legal hold	A compliance freeze that blocks destruction regardless of retention expiry.
Recertification	The vendor's periodic declaration of what they hold. The basis of inventory reconciliation.
Break / exception	What our records said on a given past date, reconstructed by replaying the event log to that timestamp.
As-of position	What our records said on a given past date, reconstructed by replaying the event log to that timestamp.